

Recursion

Lesson Objectives

By the end of this lesson, students should be able to:

- Understand what recursion is and how it works.
 - Identify **base** and **recursive** cases in algorithms.
 - Write pseudocode for common recursive problems.
 - Compare recursion and iteration.
 - Practice version control using Git for recursive solutions.
-

1 Introduction: What is Recursion?

Recursion is a programming technique where a function **calls itself** to solve smaller instances of the same problem.

It's like solving a big problem by **breaking it down** into smaller, easier subproblems.

◆ General Recursive Structure

```
PROCEDURE RecursiveFunction(parameters)
  IF base_case THEN
    RETURN simple_result
  ELSE
    RETURN RecursiveFunction(smaller_problem)
  ENDIF
ENDPROCEDURE
```

2 Base Case vs Recursive Case

- **Base Case:**

The stopping condition of the recursion. Without it, recursion continues forever (infinite loop).

- **Recursive Case:**

The step where the function calls itself to solve a smaller version of the problem.

🧠 **Think of recursion like climbing down a ladder:**

Each step takes you closer to the ground (base case).

3 Example 1: Factorial

📺 Definition:

$n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$

e.g., $4! = 4 \times 3 \times 2 \times 1 = 24$

◆ Pseudocode

```
PROCEDURE Factorial(n)
  IF n = 0 THEN
    RETURN 1    // Base case
  ELSE
    RETURN n * Factorial(n - 1) // Recursive case
  ENDIF
ENDPROCEDURE
```

◆ Dry-Run Example

```
Factorial(3)
→ 3 * Factorial(2)
→ 3 * (2 * Factorial(1))
→ 3 * (2 * (1 * Factorial(0)))
→ 3 * (2 * (1 * 1))
→ 6
```

Diagram idea:

A downward flow where each call (Factorial(3), Factorial(2), Factorial(1)) is shown as a node leading to the base case.

Example 2: Fibonacci Sequence

Definition:

$F(n) = F(n - 1) + F(n - 2)$, where
 $F(0) = 0$ and $F(1) = 1$

◆ Pseudocode

```
PROCEDURE Fibonacci(n)
  IF n = 0 THEN
    RETURN 0
  ELSE IF n = 1 THEN
    RETURN 1
  ELSE
    RETURN Fibonacci(n - 1) + Fibonacci(n - 2)
  ENDIF
ENDPROCEDURE
```

◆ Example:

```
Fibonacci(4)
= Fibonacci(3) + Fibonacci(2)
= (Fibonacci(2) + Fibonacci(1)) + (Fibonacci(1) + Fibonacci(0))
= ((1 + 0) + 1) + (1 + 0)
= 3
```

Diagram idea:

A **tree diagram** showing function calls:

```
      F(4)
     /  \
    /    \
  F(3)  F(2)
```

/ \ / \
F(2) F(1) F(1) F(0)

5 Example 3: Sum of Array Elements

◆ Pseudocode

```
PROCEDURE SumArray(A, n)
  IF n = 0 THEN
    RETURN 0
  ELSE
    RETURN A[n-1] + SumArray(A, n - 1)
  ENDIF
ENDPROCEDURE
```

◆ Example:

For A = [2, 4, 6]

```
SumArray(A, 3)
= A[2] + SumArray(A, 2)
= 6 + (4 + (2 + 0))
= 12
```



6 Comparing Recursion and Iteration

Feature	Recursion	Iteration
Concept	Function calls itself	Repeats using loops
Control	Base case	Loop condition
Memory use	Higher (stack frames)	Lower
Readability	Often clearer	Often faster
Example	Factorial using recursion	Factorial using for-loop

◆ Iterative Example (Factorial)

```
PROCEDURE FactorialIterative(n)
```

```
result ← 1
FOR i ← 1 TO n
    result ← result * i
ENDFOR
RETURN result
ENDPROCEDURE
```

7 Lab Activity

✿ Task 1:

Write pseudocode for a recursive function that counts down from n to 1.

✿ Task 2:

Convert your **Factorial recursion** into **iteration** and compare both approaches.

✿ Expected Output Example:

Input: 5
Countdown: 5, 4, 3, 2, 1

8 Git Task

1. Create a **new branch** named `recursion_solutions`.
 2. Add your recursive pseudocode files (`factorial.txt`, `fibonacci.txt`, `sumarray.txt`).
 3. Write short documentation explaining:
 - How recursion works.
 - The role of the base case.
 4. **Commit and push** your changes:
 5. `git add .`
 6. `git commit -m "Added recursive algorithms for factorial, fibonacci, and sum"`
 7. `git push origin recursion_solutions`
-